Algoritmos de optimización - Seminario

Nombre y Apellidos:
Adrian Roizman
Sebastian Liicardello

Url: https://github.com/.../03MAIR---Algoritmos-de-Optimizacion---2019/tree/master/SEMINARIO

Problema:

> 1. Sesiones de doblaje

Descripción del problema:
Se precisa coordinar el doblaje de una película. Los actores del doblaje deben coincidir en las tomas en las que sus personajes aparecen juntos en las diferentes tomas. Los actores de doblaje cobran todos la misma cantidad por cada día que deben desplazarse hasta el estudio de grabación, independientemente del número de tomas que se graben. No es posible grabar más de 6 tomas por día. El objetivo es planificar las sesiones por día de manera que el gasto por los servicios de los actores de doblaje sea el menor posible.

Los datos son: número de actores 10 y número de tomas 30. En la matriz de datos, un 1 indica que el actor participa en la toma y un 0 indica que no participa.

(*) La respuesta es obligatoria

In [2]:
```
# Configuracion inicial y carga de los datos del problema.
# La matriz procede del fichero Excel proporcionado (30 tomas x 10 actores).
# Cada fila es una toma y cada columna un actor: 1 si el actor participa, 0 si no.

import math
import random

NUM_ACTORES = 10
NUM_TOMAS = 30
MAX_TOMAS_DIA = 6

# Matriz de participacion (tomas x actores)
TOMAS = [
    [1, 1, 1, 1, 1, 0, 0, 0, 0, 0],
    [0, 0, 1, 1, 1, 0, 0, 0, 0, 0],
    [0, 1, 0, 0, 1, 0, 1, 0, 0, 0],
    [1, 1, 0, 0, 0, 0, 1, 1, 0, 0],
    [0, 1, 0, 1, 0, 0, 0, 1, 0, 0],
    [1, 1, 0, 1, 1, 0, 0, 0, 0, 0],
    [1, 1, 0, 1, 1, 0, 0, 0, 0, 0],
    [1, 1, 0, 0, 0, 1, 0, 0, 0, 0],
    [1, 1, 0, 1, 0, 0, 0, 0, 0, 0],
```

```
    [1, 1, 0, 0, 0, 1, 0, 0, 1, 0],
    [1, 1, 1, 0, 1, 0, 0, 1, 0, 0],
    [1, 1, 1, 1, 0, 1, 0, 0, 0, 0],
    [1, 0, 0, 1, 1, 0, 0, 0, 0, 0],
    [1, 0, 1, 0, 0, 1, 0, 0, 0, 0],
    [1, 1, 0, 0, 0, 0, 1, 0, 0, 0],
    [0, 0, 0, 1, 0, 0, 0, 0, 0, 1],
    [1, 0, 1, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 1, 0, 0, 1, 0, 0, 0, 0],
    [1, 0, 1, 0, 0, 0, 0, 0, 0, 0],
    [1, 0, 1, 1, 1, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 1, 0, 1, 0, 0],
    [1, 1, 1, 1, 0, 0, 0, 0, 0, 0],
    [1, 0, 1, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 1, 0, 0, 1, 0, 0, 0, 0],
    [1, 1, 0, 1, 0, 0, 0, 0, 0, 1],
    [1, 0, 1, 0, 1, 0, 0, 0, 1, 0],
    [0, 0, 0, 1, 1, 0, 0, 0, 0, 0],
    [1, 0, 0, 1, 0, 0, 0, 0, 0, 0],
    [1, 0, 0, 0, 1, 1, 0, 0, 0, 0],
    [1, 0, 0, 1, 0, 0, 0, 0, 0, 0],
]

def actores_de_toma(toma):
    # Devuelve el conjunto de actores (indices) que participan en una toma
    return {actor for actor, participa in enumerate(TOMAS[toma]) if participa == 1}

print("Numero de tomas:", NUM_TOMAS)
print("Numero de actores:", NUM_ACTORES)
print("Maximo de tomas por dia:", MAX_TOMAS_DIA)
print("Dias minimos necesarios:", math.ceil(NUM_TOMAS / MAX_TOMAS_DIA))
```

```
Numero de tomas: 30
Numero de actores: 10
Maximo de tomas por dia: 6
Dias minimos necesarios: 5
```

(*)¿Cuantas posibilidades hay sin tener en cuenta las restricciones?

¿Cuantas posibilidades hay teniendo en cuenta todas las restricciones.

Respuesta

Una solución consiste en repartir las 30 tomas en días de grabación. Como no se pueden grabar más de 6 tomas por día, el número mínimo de días es techo(30/6) = 5. Trabajamos por tanto con 5 días de 6 tomas cada uno.

Sin tener en cuenta las restricciones, si consideramos todas las ordenaciones posibles de las 30 tomas repartidas en las posiciones disponibles, hay 30! ordenaciones, es decir 265252859812191058636308480000000 (aproximadamente 2,65 por 10 elevado a 32).

Teniendo en cuenta las restricciones, dos observaciones reducen ese número. En primer lugar, el orden de las tomas dentro de un mismo día no influye en el coste, lo que introduce 6! = 720 permutaciones internas equivalentes por cada uno de los 5 días. En segundo lugar,

los cinco días son intercambiables entre sí, ya que el coste no depende de qué día llamemos día 1, lo que añade 5! = 120 reordenaciones equivalentes. Dividiendo por ambos factores, el número de planificaciones realmente distintas es:

30! dividido entre ((6!) elevado a 5 por 5!) = 11423951396577720 (aproximadamente 1,14 por 10 elevado a 16).

Aun aplicando las restricciones, el espacio de soluciones sigue siendo enorme, lo que anticipa que la fuerza bruta no será viable.

In [3]:
```
# Calculo del tamano del espacio de soluciones

dias = math.ceil(NUM_TOMAS / MAX_TOMAS_DIA)

sin_restricciones = math.factorial(NUM_TOMAS)

# Con restricciones: el orden dentro del dia no cuenta (6! por dia)
# y los dias son intercambiables (5!)
con_restricciones = math.factorial(NUM_TOMAS) // (
    math.factorial(MAX_TOMAS_DIA) ** dias * math.factorial(dias)
)

print("Posibilidades sin restricciones (30!):", sin_restricciones)
print("Posibilidades con restricciones:", con_restricciones)
```

```
Posibilidades sin restricciones (30!): 265252859812191058636308480000000
Posibilidades con restricciones: 11423951396577720
```

Modelo para el espacio de soluciones
(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, arguentalo)

Respuesta

Una solución (planificación) es una partición de las 30 tomas en grupos, donde cada grupo es un día y contiene como mucho 6 tomas. La estructura que mejor representa esto es una lista de listas: la lista exterior recorre los días y cada lista interior contiene los índices de las tomas asignadas a ese día. La restricción de capacidad se controla de forma directa con la longitud de cada lista interior, que no puede superar 6.

Para el cálculo del coste conviene apoyarse además en conjuntos. Los actores que participan en una toma se representan como un conjunto de índices, y los actores de un día se obtienen como la unión de los conjuntos de sus tomas. Los conjuntos son la estructura idónea aquí porque la operación clave del problema es contar actores distintos por día, es decir, calcular uniones y eliminar repetidos, algo que los conjuntos resuelven de forma natural y eficiente.

Una alternativa inicial sería un vector de asignación de longitud 30 donde la posición i indica el día de la toma i. Es válida, pero para evaluar el coste habría que reagrupar por día en cada

evaluación. Por ese motivo se prefiere la representación como lista de días apoyada en conjuntos de actores, que es más directa para la función objetivo.

In [4]:
```python
# La planificacion se modela como una lista de dias.
# Cada dia es una lista de indices de tomas. Ejemplo con una asignacion cualquiera:

ejemplo_plan = [
    [0, 1, 2, 3, 4, 5],
    [6, 7, 8, 9, 10, 11],
    [12, 13, 14, 15, 16, 17],
    [18, 19, 20, 21, 22, 23],
    [24, 25, 26, 27, 28, 29],
]

def actores_de_dia(dia):
    # Union de los actores de todas las tomas del dia (actores distintos)
    actores = set()
    for toma in dia:
        actores |= actores_de_toma(toma)
    return actores

for i, dia in enumerate(ejemplo_plan, 1):
    actores = actores_de_dia(dia)
    print("Dia", i, "-> tomas", [t + 1 for t in dia], "actores distintos:", len(act
```

```
Dia 1 -> tomas [1, 2, 3, 4, 5, 6] actores distintos: 7
Dia 2 -> tomas [7, 8, 9, 10, 11, 12] actores distintos: 8
Dia 3 -> tomas [13, 14, 15, 16, 17, 18] actores distintos: 8
Dia 4 -> tomas [19, 20, 21, 22, 23, 24] actores distintos: 7
Dia 5 -> tomas [25, 26, 27, 28, 29, 30] actores distintos: 8
```

Según el modelo para el espacio de soluciones
(*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

Respuesta

La función objetivo es el gasto total en actores. Como cada actor cobra lo mismo por cada día que acude al estudio, el coste de un día es el número de actores distintos que participan en alguna de las tomas grabadas ese día. El coste total de una planificación es la suma de los costes de todos sus días:

coste(P) = suma para cada día d de la cantidad de actores distintos que intervienen en el día d.

Se trata de un problema de minimización, ya que buscamos la planificación cuyo coste total sea el menor posible. Agrupar en un mismo día tomas que comparten actores reduce el número de desplazamientos y, por tanto, el coste.

In [5]:
```python
# Funcion objetivo: coste total de una planificacion
```

```
def coste(plan):
    total = 0
    for dia in plan:
        total += len(actores_de_dia(dia))
    return total

# Comprobacion sobre el ejemplo anterior
print("Coste del plan de ejemplo:", coste(ejemplo_plan))
```

```
Coste del plan de ejemplo: 38
```

Diseña un algoritmo para resolver el problema por fuerza bruta

Respuesta

El algoritmo por fuerza bruta genera todas las particiones posibles de las tomas en días de a lo sumo 6 tomas, calcula el coste de cada partición y se queda con la de menor coste. De esta forma se garantiza encontrar el óptimo, ya que se examina el espacio de soluciones completo.

La generación se implementa con retroceso (backtracking). Se recorren las tomas una a una y, para cada toma, se prueba a colocarla en cada día ya abierto que tenga hueco o bien a abrir un día nuevo. Cuando todas las tomas están colocadas, se evalúa el coste de la planificación y se actualiza la mejor solución encontrada.

Dado que el número de particiones de 30 tomas es del orden de 10 elevado a 16, este algoritmo solo es aplicable a instancias muy pequeñas. En la celda siguiente se ejecuta sobre un subconjunto reducido para comprobar que funciona correctamente.

In [6]:
```
# Algoritmo por fuerza bruta (busqueda exhaustiva con retroceso).
# Explora todas las particiones en dias de a lo sumo max_por_dia tomas.
# Garantiza el optimo, pero solo es viable para pocas tomas.

def fuerza_bruta(tomas_indices, max_por_dia):
    mejor = {"coste": None, "plan": None}

    def backtrack(i, plan_actual):
        if i == len(tomas_indices):
            c = coste(plan_actual)
            if mejor["coste"] is None or c < mejor["coste"]:
                mejor["coste"] = c
                mejor["plan"] = [dia[:] for dia in plan_actual]
            return
        toma = tomas_indices[i]
        # Opcion 1: colocar la toma en un dia ya abierto con hueco
        for dia in plan_actual:
            if len(dia) < max_por_dia:
                dia.append(toma)
                backtrack(i + 1, plan_actual)
                dia.pop()
        # Opcion 2: abrir un dia nuevo con esta toma
        plan_actual.append([toma])
        backtrack(i + 1, plan_actual)
```

```
        plan_actual.pop()

    backtrack(0, [])
    return mejor["plan"], mejor["coste"]

# Prueba sobre una instancia pequena (primeras 8 tomas, maximo 3 por dia)
plan_fb, coste_fb = fuerza_bruta(list(range(8)), 3)
print("Mejor plan (8 tomas, max 3 por dia):")
for i, dia in enumerate(plan_fb, 1):
    print("  Dia", i, "->", [t + 1 for t in dia])
print("Coste optimo de la instancia pequena:", coste_fb)
```

```
Mejor plan (8 tomas, max 3 por dia):
  Dia 1 -> [1, 2, 3]
  Dia 2 -> [4, 5, 8]
  Dia 3 -> [6, 7]
Coste optimo de la instancia pequena: 16
```

Calcula la complejidad del algoritmo por fuerza bruta

Respuesta

La complejidad viene dominada por el número de particiones que hay que generar. Fijando 5 días de 6 tomas, ese número es del orden de 30! dividido entre ((6!) elevado a 5 por 5!), aproximadamente 1,14 por 10 elevado a 16. Si se permite un número variable de días, la cantidad de particiones crece todavía más (relacionada con los números de Bell). Además, evaluar el coste de cada planificación cuesta del orden de n por m operaciones, siendo n el número de tomas y m el número de actores.

Se trata por tanto de una complejidad de tipo factorial o exponencial, que crece de forma inabordable con n. Tal como se estudia en la asignatura, los órdenes factoriales solo son prácticos para valores muy pequeños de n. Para 30 tomas el algoritmo es inaplicable, lo que justifica buscar una alternativa más eficiente.

In [7]:
```
# Ilustracion del crecimiento del numero de particiones en 5 dias de 6 tomas.
# Se observa como pasa de valores manejables a valores inabordables.

def num_particiones(n_tomas, tam_dia):
    d = n_tomas // tam_dia
    return math.factorial(n_tomas) // (math.factorial(tam_dia) ** d * math.factoria

for n in [6, 12, 18, 24, 30]:
    print("Tomas:", n, "-> particiones aproximadas:", num_particiones(n, 6))
```

```
Tomas: 6 -> particiones aproximadas: 1
Tomas: 12 -> particiones aproximadas: 462
Tomas: 18 -> particiones aproximadas: 2858856
Tomas: 24 -> particiones aproximadas: 96197645544
Tomas: 30 -> particiones aproximadas: 11423951396577720
```

(*)Diseña un algoritmo que mejore la complejidad del algortimo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Respuesta

Se propone un algoritmo voraz (greedy) constructivo. La idea, siguiendo la técnica voraz vista en la asignatura, es construir la solución por etapas eligiendo en cada paso la mejor opción local. El procedimiento es el siguiente. Se abre un día con una toma semilla (se elige como semilla la toma con más actores, que es la más difícil de acomodar). A continuación se va completando ese día añadiendo, entre las tomas pendientes, aquella que aporta menos actores nuevos respecto a los que ya están citados ese día, es decir, la que más solapa con el reparto actual. Cuando el día alcanza las 6 tomas, se cierra y se abre otro. El proceso continúa hasta agotar las tomas.

Para reducir la dependencia de la semilla inicial se aplica un esquema de multiarranque: se repite la construcción probando distintas tomas como primera semilla y se conserva la mejor planificación obtenida.

Este algoritmo mejora a la fuerza bruta porque no recorre el espacio de soluciones completo, sino que construye una única solución por etapas. Pasa de una complejidad factorial a una complejidad polinómica, lo que permite resolver la instancia de 30 tomas de forma casi instantánea. Como es propio de los algoritmos voraces, no garantiza el óptimo global, pero proporciona una buena solución en un tiempo muy reducido, que es justo lo que interesa cuando el espacio de búsqueda es demasiado grande.

In [8]:
```python
# Algoritmo voraz constructivo.
# En cada dia se anade la toma que menos actores nuevos aporta (mayor solapamiento)

def voraz(orden_semillas=None, max_por_dia=MAX_TOMAS_DIA):
    pendientes = list(range(NUM_TOMAS))
    plan = []
    usar_semilla_fija = orden_semillas is not None
    while pendientes:
        # Semilla del dia
        if usar_semilla_fija and orden_semillas in pendientes:
            semilla = orden_semillas
            usar_semilla_fija = False
        else:
            semilla = max(pendientes, key=lambda t: len(actores_de_toma(t)))
        dia = [semilla]
        actores_dia = set(actores_de_toma(semilla))
        pendientes.remove(semilla)
        # Completar el dia con las tomas que menos actores nuevos aporten
        while len(dia) < max_por_dia and pendientes:
            elegida = min(
                pendientes,
                key=lambda t: len(actores_de_toma(t) - actores_dia)
            )
            dia.append(elegida)
            actores_dia |= actores_de_toma(elegida)
            pendientes.remove(elegida)
        plan.append(sorted(dia))
    return plan
```

```
def voraz_multiarranque(max_por_dia=MAX_TOMAS_DIA):
    # Prueba cada toma como semilla inicial y devuelve la mejor planificacion
    mejor_plan = None
    mejor_coste = None
    for semilla in range(NUM_TOMAS):
        plan = voraz(orden_semillas=semilla, max_por_dia=max_por_dia)
        c = coste(plan)
        if mejor_coste is None or c < mejor_coste:
            mejor_coste = c
            mejor_plan = plan
    return mejor_plan, mejor_coste

plan_voraz, coste_voraz = voraz_multiarranque()
print("Planificacion voraz (multiarranque):")
for i, dia in enumerate(plan_voraz, 1):
    print("  Dia", i, "-> tomas", [t + 1 for t in dia], "actores:", len(actores_de_
print("Coste total:", coste_voraz)
```

```
Planificacion voraz (multiarranque):
  Dia 1 -> tomas [5, 9, 16, 25, 28, 30] actores: 5
  Dia 2 -> tomas [1, 2, 6, 7, 13, 17] actores: 5
  Dia 3 -> tomas [3, 4, 11, 15, 19, 23] actores: 6
  Dia 4 -> tomas [8, 12, 14, 18, 22, 24] actores: 5
  Dia 5 -> tomas [10, 20, 21, 26, 27, 29] actores: 8
Coste total: 29
```

(*)Calcula la complejidad del algoritmo

Respuesta

En la versión voraz simple se colocan las n tomas una a una. Para cada toma que se coloca se recorren las tomas pendientes, del orden de n, y para cada una se calcula una diferencia de conjuntos sobre m actores, del orden de m operaciones. Esto da una complejidad del orden de n al cuadrado por m.

La versión con multiarranque repite la construcción probando cada una de las n tomas como semilla inicial, de modo que la complejidad total es del orden de n al cubo por m.

Para n = 30 tomas y m = 10 actores esto supone del orden de unos cientos de miles de operaciones elementales, un coste despreciable en la práctica. La diferencia con la fuerza bruta es enorme, ya que pasamos de un orden factorial, inabordable, a un orden polinómico que se ejecuta de forma inmediata.

In [9]:
```
# Comprobacion empirica del tiempo de ejecucion del algoritmo voraz.
import time

inicio = time.perf_counter()
_, c = voraz_multiarranque()
fin = time.perf_counter()

print("Coste obtenido:", c)
```

```
print("Operaciones estimadas del orden de n^3 * m =", NUM_TOMAS ** 3 * NUM_ACTORES)
print("Tiempo de ejecucion (segundos):", round(fin - inicio, 5))
```

```
Coste obtenido: 29
Operaciones estimadas del orden de n^3 * m = 270000
Tiempo de ejecucion (segundos): 0.01225
```

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Respuesta

Se genera una matriz binaria aleatoria de tamaño número de tomas por número de actores. Cada casilla vale 1 con una probabilidad p (por ejemplo 0,3), que controla cuántos actores intervienen de media en cada toma. Para que los datos tengan sentido se garantiza que ninguna toma quede vacía, es decir, que toda toma tenga al menos un actor asignado.

La función es parametrizable en número de tomas, número de actores y probabilidad, lo que permite generar instancias de distinto tamaño para probar el comportamiento del algoritmo y su escalabilidad.

In [10]:
```
# Generador de juegos de datos aleatorios

def genera_datos(n_tomas, n_actores, prob=0.3, semilla=None):
    if semilla is not None:
        random.seed(semilla)
    matriz = []
    for _ in range(n_tomas):
        fila = [1 if random.random() < prob else 0 for _ in range(n_actores)]
        # Garantizamos que la toma tenga al menos un actor
        if sum(fila) == 0:
            fila[random.randrange(n_actores)] = 1
        matriz.append(fila)
    return matriz

datos_aleatorios = genera_datos(30, 10, prob=0.3, semilla=42)
print("Ejemplo de juego de datos aleatorio (primeras 5 tomas):")
for i, fila in enumerate(datos_aleatorios[:5], 1):
    print("  Toma", i, "->", fila)
```

```
Ejemplo de juego de datos aleatorio (primeras 5 tomas):
  Toma 1 -> [0, 1, 1, 1, 0, 0, 0, 1, 0, 1]
  Toma 2 -> [1, 0, 1, 1, 0, 0, 1, 0, 0, 1]
  Toma 3 -> [0, 0, 0, 1, 0, 0, 1, 1, 0, 0]
  Toma 4 -> [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
  Toma 5 -> [1, 1, 1, 1, 1, 1, 1, 0, 0, 0]
```

Aplica el algoritmo al juego de datos generado

Respuesta

Se aplica el algoritmo voraz con multiarranque, primero sobre los datos reales del problema y después sobre un juego de datos aleatorio. En ambos casos se muestra el reparto de

tomas por día y el coste total obtenido. Para poder reutilizar las funciones con cualquier matriz, se define una función auxiliar que fija la matriz de trabajo y lanza la resolución.

In [11]:
```
# Aplicacion del algoritmo a una matriz cualquiera.
# Reutilizamos las funciones fijando las variables globales de trabajo.

def resolver(matriz, max_por_dia=6):
    global TOMAS, NUM_TOMAS, NUM_ACTORES, MAX_TOMAS_DIA
    TOMAS = matriz
    NUM_TOMAS = len(matriz)
    NUM_ACTORES = len(matriz[0])
    MAX_TOMAS_DIA = max_por_dia
    return voraz_multiarranque(max_por_dia)

# 1) Datos reales del problema
matriz_real = [fila[:] for fila in [
    [1,1,1,1,1,0,0,0,0,0],[0,0,1,1,1,0,0,0,0,0],[0,1,0,0,1,0,1,0,0,0],
    [1,1,0,0,0,0,1,1,0,0],[0,1,0,1,0,0,0,1,0,0],[1,1,0,1,1,0,0,0,0,0],
    [1,1,0,1,1,0,0,0,0,0],[1,1,0,0,0,1,0,0,0,0],[1,1,0,1,0,0,0,0,0,0],
    [1,1,0,0,0,1,0,0,1,0],[1,1,1,0,1,0,0,1,0,0],[1,1,1,1,0,1,0,0,0,0],
    [1,0,0,1,1,0,0,0,0,0],[1,0,1,0,0,1,0,0,0,0],[1,1,0,0,0,0,1,0,0,0],
    [0,0,0,1,0,0,0,0,0,1],[1,0,1,0,0,0,0,0,0,0],[0,0,1,0,0,1,0,0,0,0],
    [1,0,1,0,0,0,0,0,0,0],[1,0,1,1,1,0,0,0,0,0],[0,0,0,0,0,1,0,1,0,0],
    [1,1,1,1,0,0,0,0,0,0],[1,0,1,0,0,0,0,0,0,0],[0,0,1,0,0,1,0,0,0,0],
    [1,1,0,1,0,0,0,0,0,1],[1,0,1,0,1,0,0,0,1,0],[0,0,0,1,1,0,0,0,0,0],
    [1,0,0,1,0,0,0,0,0,0],[1,0,0,0,1,1,0,0,0,0],[1,0,0,1,0,0,0,0,0,0],
]]

plan_real, coste_real = resolver(matriz_real)
print("RESULTADO SOBRE LOS DATOS REALES")
for i, dia in enumerate(plan_real, 1):
    print("  Dia", i, "-> tomas", [t + 1 for t in dia], "actores:", len(actores_de_
print("Coste total (datos reales):", coste_real)
print()

# 2) Juego de datos aleatorio
matriz_rnd = genera_datos(30, 10, prob=0.3, semilla=42)
plan_rnd, coste_rnd = resolver(matriz_rnd)
print("RESULTADO SOBRE UN JUEGO DE DATOS ALEATORIO")
for i, dia in enumerate(plan_rnd, 1):
    print("  Dia", i, "-> tomas", [t + 1 for t in dia], "actores:", len(actores_de_
print("Coste total (datos aleatorios):", coste_rnd)
```

```
RESULTADO SOBRE LOS DATOS REALES
  Dia 1 -> tomas [5, 9, 16, 25, 28, 30] actores: 5
  Dia 2 -> tomas [1, 2, 6, 7, 13, 17] actores: 5
  Dia 3 -> tomas [3, 4, 11, 15, 19, 23] actores: 6
  Dia 4 -> tomas [8, 12, 14, 18, 22, 24] actores: 5
  Dia 5 -> tomas [10, 20, 21, 26, 27, 29] actores: 8
Coste total (datos reales): 29

RESULTADO SOBRE UN JUEGO DE DATOS ALEATORIO
  Dia 1 -> tomas [4, 8, 15, 17, 18, 22] actores: 5
  Dia 2 -> tomas [2, 5, 11, 13, 20, 24] actores: 8
  Dia 3 -> tomas [1, 3, 10, 19, 23, 30] actores: 6
  Dia 4 -> tomas [6, 9, 14, 16, 25, 28] actores: 8
  Dia 5 -> tomas [7, 12, 21, 26, 27, 29] actores: 9
Coste total (datos aleatorios): 36
```

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

Respuesta

1. Material de la asignatura Algoritmos de optimización, en particular los apartados sobre análisis de la complejidad, órdenes de complejidad y algoritmos voraces.
2. Brassard, G. y Bratley, P. (1997). Fundamentos de algoritmia. Prentice Hall.
3. Guerequeta, R. y Vallecillo, A. (2000). Técnicas de diseño de algoritmos. Universidad de Málaga.
4. Documentación oficial de Python para los módulos math.
5. Herramienta de inteligencia artificial como apoyo en la implementación del código en Python.

Describe brevemente las lineas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Respuesta

Una primera línea de avance es añadir una fase de mejora local sobre la solución voraz, por ejemplo intercambiando tomas entre días si con ello se reduce el coste, lo que suele acercar bastante más la solución al óptimo sin cambiar el orden polinómico.

Una segunda línea es aplicar métodos exactos que sí garanticen el óptimo para tamaños moderados, como la ramificación y poda (branch and bound) o el retroceso con poda, descartando ramas que no puedan mejorar la mejor solución encontrada. También cabría una formulación como programación lineal entera.

Para tamaños grandes serían apropiadas las metaheurísticas vistas en la asignatura, como el recocido simulado, los algoritmos genéticos o los procedimientos GRASP, que exploran el espacio de soluciones de forma más amplia partiendo de la solución voraz como punto de arranque.

En cuanto a variaciones del problema, se pueden considerar un número máximo de tomas por día distinto, disponibilidad limitada de cada actor en ciertos días, costes distintos por actor o restricciones de precedencia entre tomas. Todas estas variantes aumentan el interés del estudio y refuerzan la necesidad de técnicas heurísticas cuando el tamaño del problema crece.